

MACHINE LEARNING IN FINANCE – WEEK 2

- (1) **(3 Points) Testing GD:** Use the gradient descent algorithm to find the minimum of the functions:

$$f_1(x) := |x - b|^{1.5},$$

$$f_2(x) := |x - b|^{0.5},$$

$$f_3(x) := |x - b|^3,$$

where b is chosen normal with mean and variance equal to 1. You can either use the numerical derivative, or better, use the corresponding correct derivative.

Does it work? If it doesn't for some of the functions. What is the problem?

- (2) **(3 Points) Implementing batch sizes:** Use the network optimisation algorithm 'GDforRegression.py'. However, organise the gradient descent steps in so-called batches with batch-size 'batch_size'= 50. That means instead of using the entire data x, y to calculate the gradient you do the following:

- (a) shuffle the x data and use the same shuffling for y . For example if

$$x = (1, 4, 5) \quad \text{and} \quad y = (0, 2, 3)$$

you might end up with $x_{shuffle} = (1, 5, 4)$ (last two entries swapped) and apply this to y to obtain:

$$x_{shuffle} = (1, 5, 4) \quad \text{and} \quad y_{shuffle} = (0, 3, 2)$$

- (b) Take the first 'batch_size' entries from $x_{shuffle}$, $y_{shuffle}$ and use them for calculating the gradient instead of using all the data.
(c) Repeat with the next 'batch_size' entries until you have gone through all the data.

For example: If x, y have $K = 200$ entries each and we use the 'batch_size'= 50, then *one* gradient descent step is replaced by $4 = K/\text{'batch_size'}$ mini-steps where each step uses 'batch_size'= 50 of the data pairs (x_k, y_k) and updates A and b accordingly each time.